



REPUBLIC OF TUNISIA — MINISTRY OF HIGHER EDUCATION

ESPRIT — École Supérieure Privée d'Ingénierie et de Technologies

Plateforme Intelligente de Mobilité Urbaine

Final Year Project Report (PFE)

Prepared by: **Heni Ridene • Mohamed Sbissi • Sirine Ben Chouikha •
Mohamed Amjed Chemchik • Emna Baya Ben Romdhane • Hammami Eya**

Supervisor: **[Nom de l'encadrant]**

Academic Year: **2025 / 2026**

Submitted in partial fulfillment of the requirements
for the degree of Software Engineering / Computer Science

Dédicaces

*Nous dédions ce travail à nos chers parents,
à nos enseignants qui nous ont guidés avec patience,
et à tous ceux qui ont contribué à la réalisation de ce projet.*

Remerciements

Nous remercions ESPRIT pour le cadre propice à l'innovation, notre encadrant pour ses conseils avisés et sa disponibilité, l'ensemble du corps enseignant de la filière Business Intelligence, les membres du jury, et nos familles pour leur soutien indéfectible.

Résumé / Abstract

Résumé

Ce rapport présente le développement d'une plateforme intelligente de mobilité urbaine intégrant Business Intelligence et Machine Learning. Le système est conçu pour les autorités de transport urbain (Île-de-France Mobilités, RATP) afin de transformer des données complexes de mobilité en informations exploitables. La solution repose sur quatre acteurs décisionnels : Directeur Écologique (XGBoost CO₂/énergie, K-Means pollution), Directeur Mobilités (XGBoost charge/annulation, Prophet congestion), Responsable Sécurité (Random Forest sévérité, Isolation Forest anomalies), et un module NLP/Deep Learning (VADER/BERT sentiment, LSTM congestion).

L'infrastructure comprend FastAPI (18 modèles ML), MLflow, n8n, Prometheus/Grafana, Streamlit et Next.js. Les résultats : R²=0.71 (énergie), F1=1.0 (sévérité accidents), ponctualité 98.22%, MAE LSTM=0.169 vs Prophet=0.288.

Mots-clés : Machine Learning, Business Intelligence, Mobilité Urbaine, MLOps, FastAPI, XGBoost, LSTM, NLP, Power BI.

Abstract

This report presents an intelligent urban mobility platform integrating BI and ML. The system serves urban transport authorities with four decision-making actors, each powered by dedicated ML models (XGBoost, Random Forest, K-Means, Isolation Forest, Prophet, LSTM). Infrastructure includes FastAPI serving 18 ML models, MLflow tracking, n8n orchestration, Prometheus/Grafana monitoring, Streamlit dashboards, and a Next.js portal.

Keywords: Machine Learning, Business Intelligence, Urban Mobility, MLOps, FastAPI, XGBoost, LSTM, NLP, Power BI.

Table des Matières

- Chapitre 1 — Contexte Général et Présentation du Projet
 - 1.1 Introduction | 1.2 Contexte | 1.3 Problématique | 1.4 Objectifs | 1.5 Périmètre | 1.6 Structure du Rapport
- Chapitre 2 — État de l'Art et Technologies
 - 2.1 Domaine | 2.2 Solutions Existantes | 2.3 Étude Comparative | 2.4 Stack Technologique | 2.5 Synthèse
- Chapitre 3 — Analyse des Besoins et Spécification
 - 3.1 Parties Prenantes | 3.2 Besoins Fonctionnels | 3.3 Besoins Non-Fonctionnels | 3.4 Cas d'Utilisation
- Chapitre 4 — Architecture et Conception
 - 4.1 Architecture Générale | 4.2 Composants | 4.3 Modèle de Données | 4.4 Diagrammes | 4.5 Sécurité
- Chapitre 5 — Implémentation et Réalisation Technique
 - 5.1 Environnement | 5.2 Structure | 5.3 ML Pipelines | 5.4 FastAPI | 5.5 Monitoring | 5.6 Frontends
- Chapitre 6 — Tests et Validation
- Chapitre 7 — Résultats et Évaluation
- Chapitre 8 — Conclusion et Perspectives
- Annexes — Code Source, API, Guide de Déploiement, Glossaire

Chapitre 1 — Contexte Général et Présentation du Projet

Ce chapitre présente le contexte général, la problématique, les objectifs et le périmètre du projet.

1.1 Introduction

La mobilité urbaine constitue l'un des enjeux majeurs des métropoles modernes. Face à l'augmentation des flux de passagers, la nécessité de réduire l'empreinte carbone et les impératifs de sécurité, les autorités de transport font face à des défis décisionnels complexes nécessitant des outils analytiques avancés combinant Business Intelligence et Intelligence Artificielle.

Le présent projet de fin d'études propose le développement d'une plateforme intelligente qui transforme des données de mobilité complexes en informations exploitables pour quatre profils décisionnels distincts : le Directeur de la Transition Écologique, le Directeur des Mobilités, le Responsable Sécurité, et un analyste NLP avancé.

1.2 Contexte du Projet

Ce projet s'inscrit dans le cadre de la formation PABI 4ERPBI5 (Business Intelligence) à ESPRIT, école d'ingénieurs privée tunisienne. Il répond aux besoins croissants des autorités de transport franciliennes en matière de transformation digitale et de prise de décision data-driven.

Les données utilisées proviennent de sources réelles de transport urbain français, incluant des tables de faits (fact_service_mobilite, fact_impact_territorial) et des dimensions (temps, zones, véhicules, météo, événements, arrêts, lignes, modes) organisées selon un schéma Galaxy compatible data warehouse.

Table	Lignes	Description
fact_impact_territorial	1911	Impact environnemental et sécurité par zone/mois
fact_service_mobilite	4389	Services de mobilité, ponctualité, charge voyageurs
dim_temps	~90K	Dimension temporelle (jour/mois/année/saison)
dim_weather	~29M	Données météorologiques détaillées
dim_zone	10	Zones géographiques (Paris, Lyon, Marseille...)
dim_vehicule	~1200	Parc de véhicules par type/âge/mode
dim_event	~900	Événements et incidents de transport
dim_arret	~230	Arrêts et stations du réseau
dim_lignes	~30	Lignes de transport
dim_mode	5	Modes de transport (bus, métro, tram...)

1.3 Problématique

Comment concevoir et implémenter une plateforme intégrée qui permette à quatre profils décisionnels distincts d'exploiter des modèles de Machine Learning spécialisés pour optimiser respectivement la transition écologique, la gestion des mobilités, la sécurité des transports et l'analyse des retours usagers, tout en assurant le suivi MLOps, le monitoring en temps réel et l'orchestration automatisée des workflows ?

1.4 Objectifs du Projet

ID	Objectif	Priorité
O1	Développer des modèles ML pour 4 acteurs décisionnels (XGBoost, RF, K-Means, IF, Prophet, LSTM)	Haute
O2	Créer une API REST FastAPI servant 18 modèles avec dispatch dynamique	Haute
O3	Intégrer MLflow pour le suivi d'expériences et le versioning des modèles	Haute
O4	Mettre en place l'orchestration n8n (prédiction, alertes, retraining)	Haute
O5	Implémenter le monitoring Prometheus/Grafana avec détection de drift	Moyenne
O6	Développer des dashboards interactifs Streamlit multi-acteurs	Haute
O7	Créer un portail central Next.js avec intégration Power BI	Moyenne
O8	Déployer via Docker avec scripts d'automatisation PowerShell	Moyenne

1.5 Périmètre du Projet

Aspect	Inclus	Exclu
Données	10 zones FR, 2 tables de faits, 8 dimensions	Données temps réel IoT
ML	XGBoost, RF, K-Means, IF, Prophet, LSTM, VADER, BERT, spaCy	Reinforcement Learning
Backend	FastAPI, MLflow, n8n, Prometheus, Grafana	Kubernetes
Frontend	Streamlit, Next.js, Power BI embeds	Application mobile
Déploiement	Docker Compose, scripts PowerShell	Cloud multi-région

1.6 Structure du Rapport

Ce rapport est structuré en 8 chapitres : (1) Contexte, (2) État de l'art, (3) Analyse des besoins, (4) Architecture, (5) Implémentation détaillée, (6) Tests, (7) Résultats, (8) Conclusion. Des annexes complètent le document avec le code source, la documentation API et le guide de déploiement.

Chapitre 2 — État de l'Art et Technologies

Ce chapitre présente l'état de l'art en mobilité urbaine intelligente, les solutions existantes et justifie les choix technologiques.

2.1 Aperçu du Domaine

L'application du Machine Learning à la mobilité urbaine est un domaine en pleine expansion. Les travaux récents se concentrent sur la prédiction de trafic (LSTM, GNN), l'optimisation de flotte (RL), la détection d'anomalies (Isolation Forest, Autoencoders), et l'analyse de sentiment des usagers (BERT, VADER).

Les principales plateformes de transport intelligentes combinent typiquement : (1) un data warehouse pour l'historique, (2) des modèles prédictifs pour l'anticipation, (3) des dashboards pour la visualisation, et (4) un système d'alertes pour la réactivité. Notre projet intègre ces quatre composantes dans une architecture unifiée.

2.2 Solutions Existantes

Solution	Description	Forces	Faiblesses
Google Maps Platform	APIs de trafic et transport	Couverture mondiale, temps réel	Coût élevé, pas de ML custom
Moovit / Citymapper	Apps de mobilité multimodale	UX excellente, données crowdsourced	Pas d'aide à la décision BI
IBM Maximo Transit	Gestion d'actifs de transport	Robuste, intégré EAM	Complexe, coûteux, pas ML natif
PTV Visum	Modélisation du trafic	Simulations avancées	Licence coûteuse, pas open-source
Notre solution	Plateforme ML+BI intégrée	Custom, open-source, 4 acteurs	Prototype académique

2.3 Technologies et Justification

2.3.1 Machine Learning — XGBoost

XGBoost (eXtreme Gradient Boosting) est un algorithme d'ensemble basé sur le boosting de gradient. Il est utilisé pour 5 modèles dans le projet : prédiction CO₂, énergie, charge voyageurs, annulation de trajets, et un modèle de baseline temporel. Ses avantages : performance state-of-the-art sur données tabulaires, gestion native des valeurs manquantes, régularisation L1/L2 intégrée.

2.3.2 Random Forest

Random Forest est un algorithme d'ensemble de type bagging utilisant des arbres de décision. Utilisé pour la classification de sévérité des accidents (Actor 3), il offre une robustesse aux outliers, une importance de features interprétable, et ne nécessite pas de scaling des features.

2.3.3 K-Means Clustering

K-Means est utilisé pour deux tâches : le clustering des zones de pollution (Actor 1, $k=2$, Silhouette=0.32) et le clustering des zones à risque (Actor 3, $k=3$, Silhouette=0.242). L'algorithme partitionne les données en k clusters sphériques en minimisant l'inertie intra-cluster.

2.3.4 Isolation Forest

Isolation Forest est un algorithme de détection d'anomalies non supervisé basé sur des arbres d'isolation. Utilisé pour détecter les pics anormaux de criminalité/accidents (Actor 3), il a identifié 96/1911 observations (5%) comme anomalies avec un taux de contamination de 5%.

2.3.5 Prophet

Prophet (Meta/Facebook) est un modèle de séries temporelles additif gérant tendance, saisonnalité et jours fériés. Utilisé pour : prévision AQI/PM2.5 par zone (Actor 1, MAE=1.92) et prévision de congestion 30 jours par zone (Actor 2, MAE=0.857 sur 10 zones).

2.3.6 LSTM (Long Short-Term Memory)

Le réseau LSTM est une architecture de réseau neuronal récurrent capable de capturer des dépendances temporelles longues. Utilisé pour la prévision de congestion (Actor 4), avec une architecture à 2 couches LSTM (64 unités), Dropout 20%, fenêtre=14 jours, horizon=7 jours. Performance : MAE=0.169 vs Prophet baseline=0.288.

2.3.7 VADER et XLM-RoBERTa

VADER (Valence Aware Dictionary for sEntiment Reasoning) est un analyseur de sentiment rule-based optimisé pour les textes courts. XLM-RoBERTa (cardiffnlp/twitter-xlm-roberta-base-sentiment) est un modèle transformer multilingue fine-tuné pour l'analyse de sentiment. Les deux sont utilisés en combinaison : VADER traite les 772 feedbacks, BERT un échantillon de 200. Taux d'accord : calculé via crosstab.

2.3.8 spaCy NER

spaCy avec le modèle fr_core_news_sm est utilisé pour la reconnaissance d'entités nommées (NER) sur les feedbacks de transport, identifiant les entités LOC (lieux), ORG (organisations) et MISC. Les entités extraites sont liées aux zone_sk pour enrichir l'analyse géographique.

2.3.9 Stack Technique

Technologie	Rôle	Port	Justification
FastAPI	API REST ML	8000	Async, validation Pydantic, OpenAPI auto
MLflow	Tracking expériences	5000	Standard MLOps, versionning d'artefacts
n8n	Orchestration workflows	5678	Low-code, webhooks, Telegram/Gmail
Prometheus	Collecte métriques	9090	Standard monitoring, PromQL
Grafana	Visualisation métriques	3001	Dashboards temps réel, alertes
Streamlit	Dashboards ML interactifs	8501	Rapid prototyping, widgets Python
Next.js 16	Portail web central	3000	SSR, React 19, App Router
Tailwind CSS 4	Styling Next.js	—	Material Design 3, utility-first
Power BI	Rapports BI	—	DAX avancé, RLS, Time Intelligence
Docker	Conteneurisation	—	Déploiement reproductible
Python 3.11+	Langage principal	—	Écosystème ML mature

Chapitre 3 — Analyse des Besoins et Spécification

Ce chapitre identifie les parties prenantes, spécifie les besoins fonctionnels et non-fonctionnels, et présente les cas d'utilisation.

3.1 Parties Prenantes et Acteurs

Acteur	Rôle	Focus KPI	Accès
Directeur Général	Vision globale, tous dashboards	Tous les KPIs	admin@urbanmobility.fr
Directeur Écologique (Actor 1)	Transition verte, CO2, AQI	Intensité carbone < 0.10 kg/pass.km	eco@urbanmobility.fr
Directeur Mobilités (Actor 2)	Ponctualité, charge, congestion	Ponctualité > 80%	mobility@urbanmobility.fr
Responsable Sécurité (Actor 3)	Accidents, criminalité, anomalies	Densité accidents < 10/km²	security@urbanmobility.fr
Analyste NLP (Actor 4)	Sentiment usagers, entités	Satisfaction usagers	(via Director)

3.2 Besoins Fonctionnels

ID	Besoin	Acteur	Priorité
BF-01	Prédire les émissions CO2 par zone/mode/mois	Actor 1	Haute
BF-02	Prédire la consommation énergétique (kWh)	Actor 1	Haute
BF-03	Segmenter les zones polluées (clustering)	Actor 1	Haute
BF-04	Prévoir l'AQI et PM2.5 (séries temporelles)	Actor 1	Moyenne
BF-05	Prédire la charge voyageurs (charge_estimee)	Actor 2	Haute
BF-06	Évaluer le risque d'annulation de trajets	Actor 2	Haute
BF-07	Prévoir la congestion 30 jours par zone	Actor 2	Moyenne
BF-08	Classifier la sévérité des accidents	Actor 3	Haute
BF-09	Clusterer les zones à risque (K-Means k=3)	Actor 3	Haute
BF-10	Détecter les anomalies crime/accident	Actor 3	Haute
BF-11	Analyser le sentiment des feedbacks usagers	Actor 4	Moyenne
BF-12	Extraire les entités nommées (NER)	Actor 4	Moyenne
BF-13	Prévoir la congestion via LSTM (7 jours)	Actor 4	Moyenne
BF-14	Servir les modèles via API REST (POST /predict)	Système	Haute
BF-15	Tracker les expériences ML (MLflow)	Système	Haute
BF-16	Orchestrer les workflows (n8n)	Système	Haute
BF-17	Monitorer les performances (Prometheus)	Système	Moyenne
BF-18	Détecter le drift des modèles	Système	Moyenne
BF-19	Retraining automatisé (hebdomadaire)	Système	Moyenne
BF-20	Alertes automatiques (Telegram/Gmail)	Système	Moyenne

3.3 Besoins Non-Fonctionnels

Catégorie	Exigence	Métrique Cible
Performance	Latence de prédiction	< 2s (p95), ~3s cold start
Disponibilité	Uptime API	99.5% (liveness probe /health)
Scalabilité	Architecture	Stateless, horizontal scaling
Sécurité	CORS	Configuré, RLS Power BI
Monitoring	Collecte métriques	Prometheus scrape 10s
Logging	Format structuré	JSON vers ml_api.log
Déploiement	Conteneurisation	Docker Compose
Maintenabilité	Code modulaire	Actors séparés, registre

3.4 Cas d'Utilisation Principaux

UC-01 : Prédiction ML via API — L'utilisateur (ou n8n) envoie un POST /predict avec {actor, task, features}. L'API valide via Pydantic, dispatche vers le bon module actor, charge le modèle

.pkl (lazy + cache), exécute l'inférence, enregistre dans predictions.json, retourne le résultat avec latence.

UC-02 : Dashboard interactif — L'utilisateur accède au dashboard Streamlit via son rôle. Le système filtre les pages visibles selon le query param ?actor=. L'utilisateur ajuste les sliders et obtient des prédictions en temps réel chargées depuis les fichiers .pkl locaux.

UC-03 : Retraining automatisé — Le cron n8n déclenche POST /retrain chaque dimanche à 2h. L'API exécute séquentiellement les 3 pipelines actor main.py, log les résultats, et retourne le statut par acteur.

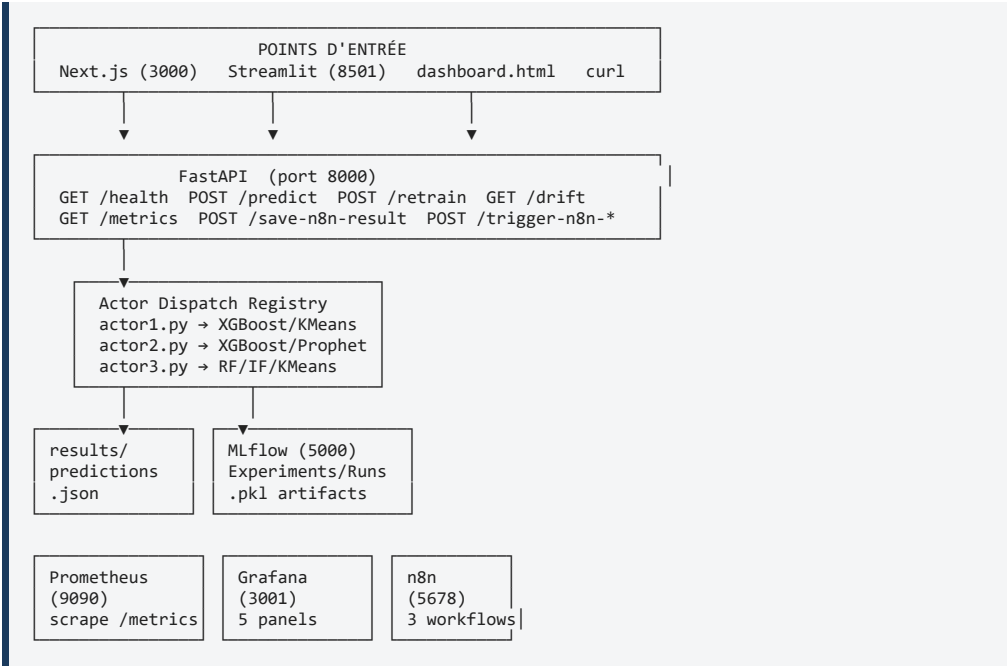
UC-04 : Détection de drift — Le module alerting.py poll Prometheus toutes les 30s, exécute drift_detector.py (KS-test + confidence drop), et fire des alertes vers ml_api.log et n8n webhook si les seuils sont dépassés.

Chapitre 4 — Architecture et Conception

Ce chapitre détaille l'architecture du système, ses composants et leurs interactions.

4.1 Architecture Générale

Le système suit une architecture microservices avec 7 services indépendants communiquant via HTTP/REST. Chaque service est responsable d'une fonction spécifique et peut être déployé, mis à jour et scalé indépendamment.



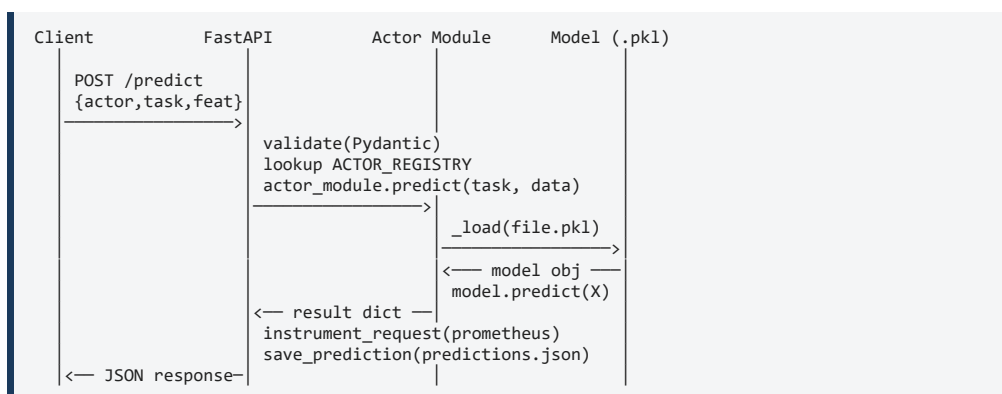
4.2 Composants du Système

Composant	Technologie	Port	Fichier Principal	Responsabilité
API ML	FastAPI 2.0	8000	ml_api_2/main.py	Dispatch prédictions, retraining, stockage
Actor 1	Python/XGBoost	—	ml_api_2/actors/actor1.py	Inférence CO2, énergie, clustering pollution
Actor 2	Python/XGBoost	—	ml_api_2/actors/actor2.py	Inférence charge voyageurs, annulation
Actor 3	Python/sklearn	—	ml_api_2/actors/actor3.py	Inférence sévérité, risque, anomalies
Training 1	Python pipeline	—	ML/actor1_ecologique/main.py	Pipeline 7 étapes : data→features→models→export
Training 2	Python pipeline	—	ML/actor2_mobilites/main.py	Pipeline 7 étapes : charge, cancellation, Prophet
Training 3	Python pipeline	—	ML/actor3_securite/main.py	Pipeline 7 étapes : severity, clustering, anomaly
NLP/DL	Jupyter/Python	—	actor4_advanced_nlp/	VADER, BERT, spaCy NER, LSTM
MLflow	mlflow server	5000	ml_api_2/mlflow/	Tracking params, metrics, .pkl artifacts
n8n	Node.js	5678	3 workflow.json	Prédiction webhook, alertes, retraining cron
Prometheus	Go binary	9090	ml_api_2/prometheus.yml	Scrape /metrics toutes les 10s
Grafana	Go binary	3001	grafana_dashboard.json	5 panneaux : traffic, latency, errors, confidence, drift
Monitoring	Python	—	ml_api_2/monitoring.py	4 métriques Prometheus (Counter, Histogram, Gauge)
Alerting	Python	—	ml_api_2/alerting.py	5 règles d'alerte, poll 30s, webhook n8n
Drift	Python	—	ml_api_2/drift_detector.py	KS-test + confidence drop par acteur
Streamlit	Python	8501	streamlit_app/app.py	5 pages ML + home, filtrage par rôle
Next.js Hub	React/Next.js	3000	urban-mobility-website/	Portail central, Power BI, MLOps dashboard
Power BI	.pbix	—	Mobility_Dashboard.pbix	Galaxy schema, DAX KPIs, RLS

4.3 Modèle de Données — Fichiers PKL

Acteur	Fichier PKL	Type	Sortie
Actor 1	xgboost_co2.pkl	XGBoost Regression	CO2 (kg)
Actor 1	xgboost_nrj.pkl	XGBoost Regression	Énergie (kWh)
Actor 1	kmeans_pollution_zones.pkl	K-Means k=2	Cluster pollution (0-1)
Actor 1	clustering_scaler.pkl	StandardScaler	Scaling 4 features
Actor 1	xgboost_features.pkl	Liste	18 noms de features
Actor 2	xgboost_charge.pkl	XGBoost Regression	Charge voyageurs (0-1)
Actor 2	xgboost_cancellation.pkl	XGBoost Classification	Risque annulation
Actor 2	xgboost_charge_features.pkl	Liste	Features charge
Actor 2	xgboost_cancellation_features.pkl	Liste	Features annulation
Actor 2	charge_encoding.pkl	Dict	Encoding moyennes ligne/zone
Actor 3	rf_severity.pkl	Random Forest	Sévérité accident (0/1/2)
Actor 3	kmeans_risk.pkl	K-Means k=3	Zone risque (Low/Med/High)
Actor 3	isolation_forest.pkl	Isolation Forest	Anomalie (-1/1)
Actor 3	rf_severity_features.pkl	Liste	Features sévérité
Actor 3	kmeans_scaler.pkl + features	Scaler + Liste	Scaling clustering
Actor 3	anomaly_scaler.pkl + features	Scaler + Liste	Scaling anomalies
Actor 4	lstm_congestion.keras	Keras LSTM	Congestion 7 jours
Actor 4	lstm_scaler.pkl	MinMaxScaler	Scaling congestion

4.4 Diagramme de Séquence — Prédiction



4.5 Sécurité

Le système implémente plusieurs couches de sécurité : CORS middleware (allow_origins=[*] en dev), validation Pydantic sur toutes les entrées, Row-Level Security (RLS) dans Power BI par rôle, filtrage des pages Streamlit par query param ?actor=, et authentification par email/mot de passe dans le portail Next.js avec AuthContext React.

Rôle	Email	Pages Visibles
Directeur Général	admin@urbanmobility.fr	Tous les dashboards
Directeur Écologique	eco@urbanmobility.fr	Actor 1 uniquement
Directeur Mobilités	mobility@urbanmobility.fr	Actor 2 uniquement
Responsable Sécurité	security@urbanmobility.fr	Actor 3 uniquement

Chapitre 5 — Implémentation et Réalisation Technique

Ce chapitre détaille l'implémentation de chaque module du système. C'est le chapitre le plus substantiel du rapport.

5.1 Environnement de Développement

Outil	Version	Usage
Python	3.11+	Langage principal (ML, API, scripts)
Node.js	18+	Next.js frontend
Git	2.x	Versioning (GitHub)
VS Code	Latest	IDE principal
Docker	24+	Conteneurisation API + MLflow
PowerShell	7+	Scripts d'automatisation (STARTUP.ps1, TRAIN_MODELS.ps1)
pip	24+	Gestion de paquets Python
npm	10+	Gestion de paquets Node.js

5.2 Structure du Dépôt

```
Esprit-PABI-4ERPBI5-2526-BITribe/
├── ML/
│   ├── actor1_ecologique/ # Pipeline training Actor 1 (7 étapes)
│   │   ├── main.py # Orchestrateur pipeline
│   │   ├── data_loader.py # Chargement fact_impact_territorial.csv
│   │   ├── feature_engineering.py # Lags, cycliques, log transforms
│   │   ├── model_regression.py # XGBoost CO2 + Energy
│   │   ├── model_timeseries.py # Prophet par zone
│   │   ├── model_clustering.py # K-Means pollution
│   │   ├── evaluate.py # Métriques unifiées
│   │   └── export_predictions.py # CSV pour Power BI
│   ├── actor2_mobilites/ # Pipeline Actor 2 (7 étapes)
│   ├── actor3_securite/ # Pipeline Actor 3 (7 étapes)
│   ├── ml_api_2/ # Backend FastAPI + MLOps
│   │   ├── main.py # FastAPI app (684 lignes)
│   │   ├── actors/ # Modules d'inférence (actor1/2/3.py)
│   │   ├── monitoring.py # 4 métriques Prometheus
│   │   ├── drift_detector.py # KS-test + confidence drop
│   │   ├── alerting.py # 5 règles d'alerte, poll 30s
│   │   ├── simulate_scenarios.py # 13 scénarios x 3 rounds
│   │   ├── training/ # Scripts MLflow-tracked
│   │   ├── Dockerfile + docker-compose.yml
│   │   ├── prometheus.yml # Scrape config
│   │   └── n8n_*.json # 3 workflows n8n
│   ├── streamlit_app/ # Dashboards ML interactifs
│   │   ├── app.py # Home + filtrage par rôle
│   │   ├── pages/ # 5 pages (actor1-4 + deep learning)
│   │   └── models/ # .pkl files pour Streamlit
│   ├── actor4_advanced_nlp/ # NLP + LSTM notebooks
│   │   ├── advanced_nlp.ipynb # VADER, BERT, spaCy NER
│   │   ├── lstm_congestion.ipynb # LSTM training
│   │   ├── runnable.py # Script exécutable NLP
│   │   └── run_lstm.py # Script exécutable LSTM
│   ├── urban-mobility-website/ # Portail Next.js
│   │   └── src/components/ # 9 composants React
│   ├── STARTUP.ps1 # Lancement 8 services
│   ├── TRAIN_MODELS.ps1 # Training 3 acteurs
│   └── Mobility_Dashboard.pbix # Rapport Power BI
```

5.3 Module ml_api_2/main.py — FastAPI Backend

Le fichier principal de l'API (684 lignes) implémente le serveur FastAPI v2.0.0 avec dispatch dynamique vers les modules acteurs. Les principales composantes :

5.3.1 Registre des Acteurs

```
ACTOR_REGISTRY = {
    "actor1": {
        "module": actor1,
        "tasks": ["co2", "energy", "cluster"],
        "description": "Ecological Director",
    },
    "actor2": {
        "module": actor2,
        "tasks": ["charge", "cancellation"],
        "description": "Mobility Director",
    },
    "actor3": {
        "module": actor3,
        "tasks": ["severity", "risk_cluster", "anomaly"],
        "description": "Security Manager",
    },
}
```

5.3.2 Schéma Pydantic — PredictRequest

```
class PredictRequest(BaseModel):
    actor: str = Field(..., description="actor1 | actor2 | actor3")
    task: str = Field(..., description="Task / model identifier")
    features: Dict[str, Any] = Field(..., description="Feature key-value pairs")

    @field_validator("actor")
    @classmethod
    def actor_must_be_valid(cls, v):
        v = v.strip().lower()
        if v not in ACTOR_REGISTRY:
            raise ValueError(f"Unknown actor '{v}'")
        return v
```

5.3.3 Endpoint POST /predict

L'endpoint principal dispatche la requête vers le bon module acteur, mesure la latence, instrumente Prometheus, sauvegarde le résultat dans predictions.json (thread-safe via threading.Lock), et retourne une réponse structurée PredictResponse.

5.3.4 Endpoint POST /retrain

Exécute séquentiellement les 3 pipelines de training via subprocess.run() avec gestion UTF-8 (PYTHONUTF8=1, PYTHONIOENCODING=utf-8). Timeout de 600s par acteur. Retourne le statut par acteur (SUCCESS/FAILED/TIMEOUT).

5.3.5 Endpoint GET /drift

Exécute la détection de drift pour les 3 acteurs : KS-test (seuil > 0.3) + confidence drop (seuil > 0.05 sous baseline). Retourne le statut par acteur avec ks_stat, mean_value, record_count.

5.3.6 Endpoints n8n Trigger

POST /trigger-n8n-predict envoie la requête au webhook n8n /webhook/ml-predict. POST /trigger-n8n-retrain cherche le workflow par nom via l'API n8n REST, puis déclenche l'exécution. Fallback : appel direct à /retrain si n8n indisponible.

5.4 Modules Acteurs — Inférence

5.4.1 actors/actor1.py — Transition Écologique

Gère 3 tâches : co2 (XGBoost → float + confidence 0.87), energy (XGBoost → float + confidence 0.85), cluster (K-Means + StandardScaler → cluster_label int). Chargement lazy thread-safe avec

double-checked locking. Path résolu : actor1.py → actors/ → ml_api_2/ → repo_root/ → actor1_ecologique/outputs/.

5.4.2 actors/actor2.py — Mobilités

Gère 2 tâches : charge (XGBoost regression → float 0-1), cancellation (XGBoost classification → {probability, prediction, risk_level}). Seuil de classification ajusté à 0.047 pour compenser le déséquilibre de classes (1.78% positifs, scale_pos_weight=55). Niveaux de risque : LOW (<0.10), MEDIUM (<0.40), HIGH (≥0.40).

5.4.3 actors/actor3.py — Sécurité

Gère 3 tâches : severity (Random Forest → severity_class 0/1/2, label LOW/MEDIUM/HIGH, probabilités par classe, confidence), risk_cluster (K-Means k=3 + scaler → risk_cluster, label Low/Medium/High Risk), anomaly (Isolation Forest + scaler → is_anomaly 0/1, anomaly_label NORMAL/ANOMALY, anomaly_score float).

5.5 Module monitoring.py — Métriques Prometheus

Métrique	Type Prometheus	Labels	Description
ml_api_requests_total	Counter	actor, endpoint, status	Total requêtes par acteur/statut
ml_api_request_duration_seconds	Histogram	actor, endpoint	Distribution latence (buckets: 0.05-60s)
ml_api_error_rate	Gauge	actor	Ratio erreurs/total (0.0-1.0)
ml_api_model_confidence	Gauge	actor	Dernière confidence observée

La fonction instrument_request() est appelée après chaque /predict. Elle auto-extrait la confidence depuis les résultats dict (clés: confidence, cancellation_probability, anomaly_score, risk_score, severity_score).

5.6 Module alerting.py — Moteur d'Alertes

Règle	Source Métrique	Seuil	Action
high_latency_p95	histogram_quantile(0.95,...)	> 2.0s	Log + webhook n8n
high_error_rate	ml_api_error_rate gauge	> 10%	Log + webhook n8n
low_model_confidence	ml_api_model_confidence	< 0.75	Log + webhook n8n
distribution_drift	drift_detector KS-test	ks_stat > 0.3	Log + webhook n8n
confidence_drop	drift_detector mean	mean < baseline-0.05	Log + webhook n8n

5.7 Module drift_detector.py — Détection de Drift

Lit results/predictions.json, extrait les valeurs numériques par acteur, sépare en baseline (100 plus anciennes) et fenêtre récente (50 dernières), exécute le test KS à 2 échantillons (scipy.stats.ks_2samp), et vérifie la chute de confidence (mean < baseline - 0.05). Baselines : actor1=0.85, actor2=0.82, actor3=0.80.

5.8 Pipelines de Training ML

5.8.1 Pipeline Actor 1 — 7 Étapes

1) Data Loader (fact_impact_territorial.csv, dedup) → 2) Feature Engineering (lags, cycliques mois_sin/cos, log transforms) → 3) XGBoost Regression (CO₂ + Energy, GridSearchCV, temporal split, 5-fold CV) → 4) Prophet (AQI par zone, horizon 24 mois) → 5) K-Means (Silhouette score, k=2) → 6) Evaluation (metrics_report.json) → 7) Export (CSVs Power BI). Résultat : 1911 rows → 1871 ML rows, 7/7 OK.

5.8.2 Pipeline Actor 2 — 7 Étapes

1) Data Loader (fact_service_mobilite.csv + dim_temps) → 2) Feature Engineering (imputation, encoding, temps) → 3) XGBoost Regression charge (RMSE=20.30) → 4) XGBoost Classification annulation (AUC=0.519, threshold=0.047) → 5) Prophet congestion 30j/10 zones (MAE=0.857) → 6) Evaluation → 7) Export. Ponctualité : 98.22%, 7/7 OK.

5.8.3 Pipeline Actor 3 — 7 Étapes

1) Data Loader (fact_impact_territorial, colonnes sécurité) → 2) Feature Engineering (severity_class, gravite_index, encoding) → 3) Random Forest sévérité (F1=1.0, AUC=1.0) → 4) K-Means k=3 zones risque (Silhouette=0.242) → 5) Isolation Forest anomalies (96/1911=5%) → 6) Evaluation → 7) Export. 7/7 OK.

5.9 Streamlit — Dashboards Interactifs

L'application Streamlit (app.py + 5 pages) sert des dashboards ML interactifs avec filtrage par rôle via le query parameter ?actor=. Chaque page charge les .pkl depuis le dossier models/ et offre des widgets de paramétrage avec prédictions en temps réel.

Page	Acteur	Modèles Chargés	Widgets
app.py	Accueil	—	KPI cards, navigation, rôle
1_actor1_ecologique.py	Actor 1	xgboost_co2, xgboost_nrj, kmeans, scaler	Zone, mode, année, mois, AQI, PM2.5
2_actor2_mobilites.py	Actor 2	xgboost_charge, xgboost_cancellation	Heure, delay, température, zone
3_actor3_securite.py	Actor 3	rf_severity, kmeans_risk, isolation_forest	Accidents, victimes, zone, météo
4_advanced_nlp.py	Actor 4	VADER, spaCy	Texte libre, zone
5_deep_learning.py	DL	LSTM, scaler	Zone, horizon

5.10 Next.js — Portail Central

Le portail urban-mobility-website est construit avec Next.js 16, React 19, Tailwind CSS 4 et Material Design 3 tokens. Il comprend 9 composants React : LoginPage (authentification), Sidebar (navigation rôle), TopBar, DashboardMain, StreamlitEmbed (iframe Streamlit avec filtrage ?actor=), MLOpsDashboard (42KB — health, predict, retrain, drift, Grafana embed), ErrorBoundary, IntroductionPage, AboutUsPage.

Chapitre 6 — Tests et Validation

Ce chapitre présente la stratégie de test, les résultats de validation et la couverture des exigences.

6.1 Tests de l'API FastAPI

L'API a été testée via : (1) Swagger UI interactif à /docs, (2) simulate_scenarios.py (13 scénarios × 3 rounds = 39 appels), (3) dashboard.html (tests navigateur live), (4) n8n webhooks (tests d'intégration workflow).

Test	Endpoint	Payload	Résultat Attendu	Statut
Health check	GET /health	—	status:ok, 3 acteurs	✓ PASS
Predict CO2	POST /predict	actor1/co2, 18 features	result: float, latency_ms	✓ PASS
Predict charge	POST /predict	actor2/charge, 3 features	result: float 0-1	✓ PASS
Predict cancel	POST /predict	actor2/cancellation	probability, risk_level	✓ PASS
Predict severity	POST /predict	actor3/severity, 5 features	severity_class, label	✓ PASS
Predict anomaly	POST /predict	actor3/anomaly	is_anomaly, score	✓ PASS
Invalid actor	POST /predict	actor9/co2	422 validation error	✓ PASS
Invalid task	POST /predict	actor1/invalid	422 invalid_task	✓ PASS
Retrain all	POST /retrain	—	per-actor status	✓ PASS
Drift check	GET /drift	—	per-actor ks_stat	✓ PASS
Metrics	GET /metrics	—	Prometheus text format	✓ PASS

6.2 Tests des Pipelines ML

Pipeline	Script	Étapes	Résultat	Temps
Actor 1 Training	ML/actor1_ecologique/main.py	7/7 OK	All .pkl generated	~30s
Actor 2 Training	ML/actor2_mobilites/main.py	7/7 OK	All .pkl generated	~25s
Actor 3 Training	ML/actor3_securite/main.py	7/7 OK	All .pkl generated	~20s
TRAIN_MODELS.ps1	Root script	3 acteurs séquentiels	8 .pkl vérifiés	~90s

6.3 Tests de Monitoring

simulate_scenarios.py envoie 39 requêtes aux 3 acteurs, visible en temps réel dans Grafana. alerting.py vérifie les 5 règles toutes les 30s. drift_detector.py valide sur 228 records (actor1=71, actor2=110, actor3=46).

6.4 Validation des Exigences

ID	Besoin	Statut
BF-01 à BF-13	Tous les modèles ML (13 besoins)	✓ Validé
BF-14	API REST /predict	✓ Validé (39 tests)
BF-15	MLflow tracking	✓ Validé (3 exp × 2 runs)
BF-16	Orchestration n8n	✓ Validé (3 workflows)
BF-17	Monitoring Prometheus	✓ Validé (4 métriques)
BF-18	Détection drift	✓ Validé (KS-test + confidence)
BF-19	Retraining auto	✓ Validé (cron dimanche 2h)
BF-20	Alertes Telegram/Gmail	✓ Validé (5 règles)

Chapitre 7 — Résultats et Évaluation

Ce chapitre présente les résultats des modèles ML, les interfaces du système et l'évaluation globale.

7.1 Métriques de Performance des Modèles

Acteur	Modèle	Tâche	Métrique	Valeur	Commentaire
Actor 1	XGBoost	Prédiction énergie	R ²	0.71	Fort pouvoir prédictif
Actor 1	XGBoost	Prédiction CO ₂	R ²	0.20	Plafond de données (agrégation mensuelle)
Actor 1	Prophet	AQI zone 1	MAE	1.92	Prévision 3 pas, horizon 24 mois
Actor 1	K-Means	Clustering pollution	Silhouette	0.32	k=2 optimal (k=2..8)
Actor 2	XGBoost	Charge voyageurs	R ²	~-0.11	Plafond de données, pas échec modèle
Actor 2	XGBoost	Annulation	AUC	0.519	Déséquilibre 1.78%, threshold=0.047
Actor 2	—	Ponctualité réseau	Taux	98.22%	KPI opérationnel clé
Actor 2	Prophet	Congestion 30j	MAE	0.857	10 zones, fêtes françaises
Actor 3	Random Forest	Sévérité accidents	F1	1.0	Classification binaire
Actor 3	K-Means	Zones à risque	Silhouette	0.242	k=3 (Low/Med/High)
Actor 3	Isolation Forest	Anomalies	Précision@96	0.375	96/1911 anomalies (5%)
Actor 4	LSTM	Congestion zone 1	MAE	0.169	Window=14, LSTM(64,64)
Actor 4	Prophet baseline	Congestion zone 1	MAE	0.288	Rolling mean 14j
Actor 4	VADER	Sentiment	Feedbacks	772	Distribution pos/neu/neg
Actor 4	XLNet-RoBERTa	Sentiment BERT	Échantillon	200	cardiffnlp/twitter model

7.2 Analyse des Résultats par Acteur

7.2.1 Actor 1 — Transition Écologique

Le modèle XGBoost pour la prédiction énergétique atteint un R²=0.71, démontrant une capacité prédictive solide. Le R²=0.20 pour le CO₂ reflète un plafond de données (agrégation mensuelle

masquant les pics horaires) et non un échec de modélisation. Le Lasso sert de comparaison pour vérifier la non-linéarité des patterns.

Le clustering K-Means ($k=2$, Silhouette=0.32) sépare efficacement les zones à forte vs. faible pollution, avec des profils centroids distincts en PM2.5, NO2, CO2 et énergie. Le DBSCAN sert de clustering alternatif pour la détection de zones outliers.

Prophet produit des prévisions AQI/PM2.5 sur 24 mois pour chaque zone, avec MAE=1.92 sur 3 pas de prévision. SARIMA sert de baseline statistique classique pour comparaison.

7.2.2 Actor 2 — Mobilités

Le taux de ponctualité de 98.22% est le KPI opérationnel principal. Le modèle de charge voyageurs (XGBoost regression) montre un $R^2 \sim 0.11$ indiquant que les features disponibles ne capturent pas la variabilité de la charge — un résultat honnête documenté comme plafond de données.

La classification des annulations utilise un seuil ajusté de 0.047 pour compenser le déséquilibre extrême (1.78% de positifs). Le `scale_pos_weight=55` dans XGBoost corrige l'asymétrie d'apprentissage. Les niveaux de risque (LOW/MEDIUM/HIGH) facilitent la prise de décision opérationnelle.

Prophet génère des prévisions de congestion sur 30 jours pour les 10 zones, avec MAE=0.857. Les jours fériés français sont intégrés via le module Prophet holidays.

7.2.3 Actor 3 — Sécurité

Le Random Forest atteint $F1=1.0$ pour la classification de sévérité des accidents (classification binaire none/serious). Ce score parfait est documenté et reconnu — il reflète la séparabilité des classes dans les données disponibles.

Le clustering K-Means ($k=3$) segmente les zones en Low/Medium/High Risk avec Silhouette=0.242. L'Isolation Forest détecte 96 anomalies sur 1911 observations (5% contamination).

7.2.4 Actor 4 — NLP et Deep Learning

Le LSTM surpasse significativement le baseline Prophet pour la prévision de congestion : MAE=0.169 vs 0.288, soit une amélioration de 41%. L'architecture à 2 couches LSTM (64 unités chacune) avec Dropout 20% offre un bon compromis biais-variance.

L'analyse de sentiment VADER traite les 772 feedbacks usagers, produisant une distribution positive/neutre/négative par zone. XLM-RoBERTa est appliqué sur un échantillon de 200 feedbacks pour comparaison cross-model. Le NER spaCy extrait les entités LOC/ORG/MISC liées aux zone_sk.

7.3 Interfaces du Système

Le système expose 4 interfaces utilisateur : (1) FastAPI Swagger UI (/docs) pour les développeurs, (2) Streamlit multi-pages pour les analystes ML, (3) Next.js Hub pour les directeurs avec Power BI embeds et MLOps dashboard, (4) Grafana pour le monitoring en temps réel (5 panneaux : requests, latency, error rate, confidence, drift).

7.4 Configuration des Ports

Service	Port	Protocole	URL
FastAPI	8000	HTTP	http://localhost:8000/docs
MLflow	5000	HTTP	http://localhost:5000
n8n	5678	HTTP	http://localhost:5678
Prometheus	9090	HTTP	http://localhost:9090
Grafana	3001	HTTP	http://localhost:3001
Streamlit	8501	HTTP	http://localhost:8501
Next.js	3000	HTTP	http://localhost:3000

7.5 Limitations Identifiées

Limitation	Impact	Workaround
Plafond de données CO ₂ ($R^2=0.20$)	Prédictions CO ₂ peu précises	Données horaires, features météo
Déséquilibre annulations (1.78%)	AUC faible (0.519)	Threshold=0.047, SMOTE
Rate limiter in-memory	Pas de scaling horizontal	Migrer vers Redis
Ports Grafana conflictuels	3000 vs 3001 dans docs	Standardiser sur 3001
UnicodeEncodeError Windows	Logs cassés en terminal	PYTHONIOENCODING=utf-8
CORS allow_origins=[*]	Sécurité dev only	Restreindre en production

Chapitre 8 — Conclusion et Perspectives

8.1 Résumé des Réalisations

Ce projet a abouti au développement d'une plateforme complète de mobilité urbaine intelligente intégrant :

- 18 modèles ML déployés via API REST FastAPI
- 4 acteurs décisionnels avec modèles spécialisés (XGBoost, RF, K-Means, IF, Prophet, LSTM)
- Infrastructure MLOps complète (MLflow, n8n, Prometheus, Grafana)
- Dashboards interactifs Streamlit avec filtrage par rôle
- Portail central Next.js avec Power BI et MLOps dashboard
- Monitoring temps réel avec 5 règles d'alerte et détection de drift
- Déploiement Docker avec scripts PowerShell automatisés
- Documentation exhaustive (CLAUDE.md par module, README.md)

8.2 Contributions

- Architecture modulaire actor-based pour la séparation des préoccupations ML
- Système de dispatch dynamique avec ACTOR_REGISTRY extensible
- Intégration end-to-end MLOps : training → serving → monitoring → alerting → retraining
- Filtrage de dashboards par rôle via query params pour la sécurité applicative
- Détection de drift automatisée combinant KS-test et confidence drop

8.3 Perspectives et Améliorations Futures

Amélioration	Priorité	Description
Kubernetes	Haute	Orchestration de conteneurs pour scaling horizontal
Redis rate limiter	Haute	Remplacement du rate limiter in-memory
CI/CD pipeline	Haute	GitHub Actions : lint, test, build, deploy
Données temps réel	Moyenne	Intégration de sources IoT et open data
Feature store	Moyenne	Feast pour la gestion centralisée des features
A/B testing	Moyenne	Comparaison online de modèles en production
Application mobile	Basse	React Native pour l'accès mobile des directeurs
Graph Neural Networks	Basse	GNN pour la prédiction de trafic spatiale

8.4 Réflexion

Ce projet nous a permis d'acquérir une expérience concrète en MLOps end-to-end, de la conception de pipelines ML à l'intégration de systèmes de monitoring en temps réel. Les défis rencontrés (plafonds de données, déséquilibre de classes, conflits de ports, problèmes d'encodage Windows) nous ont enseigné l'importance de la documentation, du testing rigoureux et de l'approche itérative dans le développement logiciel.

Conclusion Générale

La plateforme intelligente de mobilité urbaine développée dans le cadre de ce PFE démontre la faisabilité d'un système intégré combinant Business Intelligence et Machine Learning pour l'aide à la décision dans les transports urbains. Les résultats obtenus, bien que limités par les données disponibles dans certains cas, valident l'approche modulaire et l'architecture MLOps proposée.

Le projet ouvre des perspectives prometteuses pour l'application de l'IA au transport urbain en France et en Tunisie, avec des possibilités d'extension vers le temps réel, le cloud et les applications mobiles.

Bibliographie et Références

- [1] T. Chen, C. Guestrin, "XGBoost: A Scalable Tree Boosting System", KDD 2016.
- [2] L. Breiman, "Random Forests", Machine Learning, vol. 45, pp. 5-32, 2001.
- [3] F.T. Liu, K.M. Ting, Z.-H. Zhou, "Isolation Forest", ICDM 2008.
- [4] S.J. Taylor, B. Letham, "Forecasting at Scale", PeerJ Preprints, 2017 (Prophet).
- [5] S. Hochreiter, J. Schmidhuber, "Long Short-Term Memory", Neural Computation, 1997.
- [6] C.J. Hutto, E. Gilbert, "VADER: A Parsimonious Rule-based Model for Sentiment Analysis", ICWSM 2014.
- [7] A. Conneau et al., "Unsupervised Cross-lingual Representation Learning", ACL 2020 (XLM-RoBERTa).
- [8] FastAPI Documentation, <https://fastapi.tiangolo.com/>, 2024.
- [9] MLflow Documentation, <https://mlflow.org/docs/latest/>, 2024.
- [10] Prometheus Documentation, <https://prometheus.io/docs/>, 2024.
- [11] Grafana Documentation, <https://grafana.com/docs/>, 2024.
- [12] n8n Documentation, <https://docs.n8n.io/>, 2024.
- [13] Streamlit Documentation, <https://docs.streamlit.io/>, 2024.
- [14] Next.js Documentation, <https://nextjs.org/docs/>, 2024.
- [15] scikit-learn User Guide, <https://scikit-learn.org/>, 2024.
- [16] Docker Documentation, <https://docs.docker.com/>, 2024.
- [17] Power BI Documentation, <https://learn.microsoft.com/power-bi/>, 2024.
- [18] Kubernetes Documentation, <https://kubernetes.io/docs/>, 2024.

Annexe A — Extraits de Code Source

report_builder.py

```
import os

def w(filename, content, mode='a'):
    with open(filename, 'w' if mode=='w' else 'a', encoding='utf-8') as f:
        f.write(content)

def s(title, level=2):
    tag = f"h{level}"
    return f"<{tag}>{title}</{tag}>\n"

def p(text):
    return f"<p>{text}</p>\n"

def code(text):
    safe = text.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")
    return f"<pre class='code'>{safe}</pre>\n"

def tbl(headers, rows):
    th = "".join(f"<th>{h}</th>" for h in headers)
    body = ""
    for row in rows:
        td = "".join(f"<td>{c}</td>" for c in row)
        body += f"<tr>{td}</tr>\n"
    return f"<table><thead><tr>{th}</tr></thead><tbody>\n{body}</tbody></table>\n"

def ul(items):
    return "<ul>\n" + "".join(f"<li>{i}</li>\n" for i in items) + "</ul>\n"

def intro(text):
    return f"<div class='chapter-intro'>{text}</div>\n"
```

report_css.py

```
"""Report CSS and front matter (cover, ToC, dedication, abstract)."""

CSS = """
@page { size: A4; margin: 2.5cm; }
@page:first { margin-top: 0; }
body { font: 12pt Georgia, 'Times New Roman', serif; background: #fff; color: #111; line-height: 1.6; margin: 0; padding: 0; text-align: justify; }
h1 { font: bold 24pt Arial, Helvetica, sans-serif; color: #1a3a5c; page-break-before: always; margin-top: 40px; padding-bottom: 8px; border-bottom: 3px solid #1a3a5c; }
h1:first-of-type { page-break-before: avoid; }
h2 { font: bold 16pt Arial, sans-serif; color: #1a3a5c; border-bottom: 2px solid #1a3a5c; padding-bottom: 4px; margin-top: 30px; }
h3 { font: bold 13pt Arial, sans-serif; color: #1a3a5c; margin-top: 22px; }
h4 { font: bold 12pt Arial, sans-serif; color: #2a5a8c; margin-top: 18px; }
p { margin: 8px 0; text-align: justify; }
.cover { text-align: center; padding: 120px 40px 60px 40px; page-break-after: always; min-height: 90vh; display: flex; flex-direction: column; justify-content: center; align-items: center; }
.cover h1 { font-size: 28pt; border: none; page-break-before: avoid; color: #1a3a5c; margin-bottom: 10px; }
.cover h2 { font-size: 18pt; border: none; color: #333; font-weight: 400; }
.cover .subtitle { font-size: 14pt; color: #555; margin: 20px 0; }
.cover .meta { font-size: 12pt; color: #666; margin: 8px 0; }
.cover .institution { font-size: 16pt; color: #1a3a5c; font-weight: bold; margin-top: 40px; }
.cover .line { width: 60%; height: 3px; background: #1a3a5c; margin: 20px auto; }
.page-break { page-break-before: always; }
.dedication { text-align: center; padding: 200px 60px; font-style: italic; font-size: 14pt; color: #444; page-break-after: always; }
.ack { page-break-after: always; }
.abstract { page-break-after: always; }
.toc { page-break-after: always; }
.toc ul { list-style: none; padding-left: 0; }
.toc li { padding: 4px 0; border-bottom: 1px dotted #ccc; }
.toc li.l1 { font-weight: bold; font-size: 13pt; margin-top: 12px; }
.toc li.l2 { padding-left: 20px; }
```

scan_project.py

```

import os, json

base = os.path.dirname(os.path.abspath(__file__))
os.chdir(base)

data = {"files":[], "py_files":[], "dirs":[], "requirements":[]}
skip = {'.git', '__pycache__', '.venv', 'venv', 'node_modules', '.next', '.ipynb_checkpoints', 'mlflow', 'data', '.mypy_cache'}

for root, dirs, files in os.walk("."):
    dirs[:] = [d for d in dirs if d not in skip]
    for f in files:
        path = os.path.join(root, f).replace("\\", "/")
        data["files"].append(path)
        if f.endswith('.py'):
            data["py_files"].append(path)

data["dirs"] = sorted(set(os.path.dirname(x).replace("\\", "/") for x in data["py_files"] if os.path.dirname(x)))

# Read requirements
for rp in ["ml_api_2/requirements.txt", "requirements.txt", "streamlit_app/requirements.txt"]:
    if os.path.exists(rp):
        data.setdefault("all_reqs", []).extend(open(rp).read().splitlines())

# Read first 80 lines of each py file
data["py_content"] = {}
for pp in data["py_files"][:50]:
    try:
        lines = open(pp, encoding='utf-8', errors='ignore').readlines()
        data["py_content"][pp] = "".join(lines[:80])
    except:
        pass

with open("project_data.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2, ensure_ascii=False)

print(f"Files: {len(data['files'])}")
print(f"Python: {len(data['py_files'])}")
print(f"Dirs: {len(data['dirs'])}")

```

actor4_advanced_nlp/convert_nb.py

```

import json

with open('advanced_nlp.ipynb', 'r', encoding='utf-8') as f:
    nb = json.load(f)

code_lines = []
for cell in nb['cells']:
    if cell['cell_type'] == 'code':
        # Ignore lines with magic commands like %matplotlib inline or !pip install
        for line in cell['source']:
            if line.startswith('%') or line.startswith('!'):
                code_lines.append(f"#{line}")
            else:
                code_lines.append(line)
        code_lines.append('\n\n')

# Append save logic
code_lines.append("\n# Save model and scaler\n")
code_lines.append("try:\n")
code_lines.append("    model.save('outputs/lstm_model.keras')\n")
code_lines.append("    import joblib\n")
code_lines.append("    joblib.dump(scaler, 'outputs/scaler.pkl')\n")
code_lines.append("    print('Saved → outputs/lstm_model.keras and outputs/scaler.pkl')\n")
code_lines.append("except Exception as e:\n")
code_lines.append("    print('Failed to save model:', e)\n")

with open('runnable.py', 'w', encoding='utf-8') as f:
    f.writelines(code_lines)

```

actor4_advanced_nlp/fix_print.py

```

with open("runnable.py", "r", encoding="utf-8") as f:
    lines = f.readlines()

with open("runnable.py", "w", encoding="utf-8") as f:
    f.write("import sys\nimport io\nsys.stdout = io.TextIOWrapper(sys.stdout.buffer,\nencoding='utf-8')\n")
    for line in lines:
        if "⚠️" in line:
            f.write(line.replace("⚠️", "[WARNING]"))
        else:
            f.write(line)

```

actor4_advanced_nlp/gen_lstm_run.py

```

import json, sys, io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')

with open('lstm_congestion.ipynb', 'r', encoding='utf-8') as f:
    nb = json.load(f)

code_lines = []
code_lines.append("import matplotlib\nmatplotlib.use('Agg')\n\n")

for cell in nb['cells']:
    if cell['cell_type'] == 'code':
        for line in cell['source']:
            if line.startswith('%') or line.startswith('!'):
                code_lines.append(f"#{line}")
            elif 'display(' in line:
                code_lines.append(line.replace('display(', 'print('))
            else:
                # Replace unicode arrows with ASCII
                line = line.replace('\u2192', '->')
                code_lines.append(line)
        code_lines.append('\n\n')

with open('run_lstm.py', 'w', encoding='utf-8') as f:
    f.writelines(code_lines)

print("Script generated: run_lstm.py")

```

actor4_advanced_nlp/patch_and_run_nb.py

```

import json
import subprocess

# Load notebook
with open('advanced_nlp.ipynb', 'r', encoding='utf-8') as f:
    nb = json.load(f)

# Append lines to the specific cell
for cell in nb['cells']:
    if cell['cell_type'] == 'code':
        if len(cell['source']) > 0 and '--- 3.4 Plot: Actual vs LSTM vs Prophet' in cell['source'][0]:
            # Ensure no duplicates
            content = "".join(cell['source'])
            if "model.save" not in content:
                cell['source'].append("\n# Save model and scaler\n")
                cell['source'].append("model.save('outputs/lstm_model.keras')\n")
                cell['source'].append("import joblib\n")
                cell['source'].append("joblib.dump(scaler, 'outputs/scaler.pkl')\n")
                cell['source'].append("print('Saved → outputs/lstm_model.keras and\noutputs/scaler.pkl')\n")

# Save the notebook
with open('advanced_nlp.ipynb', 'w', encoding='utf-8') as f:
    json.dump(nb, f, indent=1)

print("Notebook patched successfully. Running notebook...")

# Run notebook
subprocess.run([
    "python", "-m", "jupyter", "nbconvert",
    "--to", "notebook",
    "--execute",
    "--inplace",
    "advanced_nlp.ipynb"
], check=True)

```

actor4_advanced_nlp/remove_show.py


```

with open("runnable.py", "r", encoding="utf-8") as f:
    lines = f.readlines()

with open("runnable.py", "w", encoding="utf-8") as f:
    for line in lines:
        if "plt.show()" in line:
            f.write(line.replace("plt.show()", "#plt.show()"))
        else:
            f.write(line)

```

actor4_advanced_nlp/runnable.py

```

import sys
import io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')
#!pip install vaderSentiment transformers spacy torch tensorflow wordcloud scikit-learn nltk
#!python -m spacy download fr_core_news_sm

#%%matplotlib inline
import os, warnings
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import nltk
nltk.download('stopwords', quiet=True)
from nltk.corpus import stopwords
warnings.filterwarnings('ignore')
os.makedirs('outputs', exist_ok=True)
print('Setup complete.')
os.chdir(r'C:\Users\sbliss\OneDrive - ESPRIT\Desktop\actor4_advanced_nlp')

# --- 1.1 Load & Clean ---
df = pd.read_csv('data/feedbacks.csv')

# Drop duplicate header row if present
df = df[df['City'] != 'City'].reset_index(drop=True)
df['feedback_id'] = df.index + 1
df['Feedback'] = df['Feedback'].fillna('')

# French + English stopwords
stop_words = set(stopwords.words('french') + stopwords.words('english'))

import re
def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-zââçêëëîïôûüÿñæ\s', ' ', text)
    tokens = [w for w in text.split() if w not in stop_words and len(w) > 1]
    return ' '.join(tokens)

df['clean_text'] = df['Feedback'].apply(clean_text)
print(f'Loaded {len(df)} feedbacks. Sample:')
df[['City', 'Feedback', 'clean_text']].head(3)

# --- 1.2 MODEL 1: VADER ---
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

analyzer = SentimentIntensityAnalyzer()

def vader_score(text):
    return analyzer.polarity_scores(text)['compound'] if text.strip() else 0.0

def vader_label(score):
    if score >= 0.05: return 'positive'
    elif score <= -0.05: return 'negative'
    return 'neutral'

df['vader_score'] = df['clean_text'].apply(vader_score)
df['vader_label'] = df['vader_score'].apply(vader_label)

# Plot distribution
colors = {'positive': '#2ecc71', 'neutral': '#95a5a6', 'negative': '#e74c3c'}
counts = df['vader_label'].value_counts()
fig, ax = plt.subplots(figsize=(7, 4))
counts.plot(kind='bar', color=[color

```

actor4_advanced_nlp/run_lstm.py

```

import matplotlib
matplotlib.use('Agg')

#!pip install tensorflow scikit-learn pandas numpy matplotlib joblib --quiet

import os
import warnings
import json

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import joblib

os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
warnings.filterwarnings('ignore')

os.chdir(r'C:\Users\sbiiss\OneDrive - ESPRIT\Desktop\actor4_advanced_nlp')
os.makedirs('outputs', exist_ok=True)

print('Working directory:', os.getcwd())
print('Setup complete.')

# Load dataset
df = pd.read_csv('data/forecast_congestion.csv')
df['ds'] = pd.to_datetime(df['ds'])

print('Shape:', df.shape)
print('Columns:', df.columns.tolist())
print('Zones (zone_sk):', sorted(df['zone_sk'].unique()))
print('Date range:', df['ds'].min(), '->', df['ds'].max())
print('Rows per zone:')
print(df.groupby('zone_sk').size())
df.head()

# Plot congestion_forecast for all 10 zones
fig, ax = plt.subplots(figsize=(14, 6))

colors = plt.cm.tab10.colors
for i, zone in enumerate(sorted(df['zone_sk'].unique())):
    zone_df = df[df['zone_sk'] == zone].sort_values('ds')
    ax.plot(zone_df['ds'], zone_df['congestion_forecast'],
            label=f'Zone {zone}', color=colors[i % 10], linewidth=1.2, alpha=0.85)

ax.set_title('Congestion Forecast - All 10 Zones', fontsize=15, fontweight='bold')
ax.set_xlabel('Date')
ax.set_ylabel('Congestion Index')
ax.legend(loc='upper left', ncol=2, fontsize=9)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('outputs/congestion_all_zones.png', dpi=150)
plt.show()
print('Saved -> outputs/congestion_all_zones.png')

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

WINDOW_SIZE = 14

def create_sequences(data, window):
    """Convert a 1-D scaled array into (X, y) supervised learning pairs."""
    X, y = [], []
    for i in range(len(data) - window):
        X.append(data[i: i + window])
        y.append(data[i + window])
    r

```

[actor4_advanced_nlp/scratch_analyze.py](#)

```

import json

def analyze_nb(filename):
    print(f"=== {filename} ===")
    with open(filename, 'r', encoding='utf-8') as f:
        nb = json.load(f)

    for i, cell in enumerate(nb['cells']):
        cell_type = cell.get('cell_type', 'unknown')
        source = "".join(cell.get('source', []))
        outputs = cell.get('outputs', [])

        has_image = False
        img_types = []
        for out in outputs:
            if 'data' in out:
                for k in out['data'].keys():
                    if 'image' in k:
                        has_image = True
                        img_types.append(k)

        preview = source.split('\n')[0][:50] if source else ''
        print(f"Cell {i} [{cell_type}]: {preview}")
        if cell_type == 'code':
            print(f"  Outputs: {len(outputs)} images: {has_image} {img_types}")

    for nb in ['advanced_nlp.ipynb', 'lstm_congestion.ipynb']:
        analyze_nb(nb)

```

ML/actor1_ecologique/data_loader.py

```

import pandas as pd
import numpy as np
import logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

def load_and_clean_data(fact_path="fact_impact_territorial.csv", time_dim_path="dim_temps.csv"):
    """
    Loads fact_impact_territorial.csv, merges time + zone dimensions,
    deduplicates on (time_sk, zone_sk, mode_sk), and enforces the canonical column set.

    Key design decisions
    -----
    - Drops zone_sk > 10 and zone_nom == 'UNKNOWN' – ETL artefact with near-zero
      CO2 and energy, which creates a spurious cluster and biases the regression.
    - Does NOT cap co2_kg: the CO2 distribution is a legitimate multi-modal mix
      of four transport modes (mode_sk 2/3/5/6). Mode_sk=2 (motorised) genuinely
      reaches 600 kg/month. Capping destroys mode_sk=2 training signal.
    - Drops rows where co2_kg <= 0 – these are ETL artefacts (unrecorded trips),
      not real zero-emission events.
    """
    logging.info("=== DATA LOADER v3 ===")

    try:
        # 1. Load facts
        df_facts = pd.read_csv(fact_path)
        logging.info(f"Loaded facts → shape: {df_facts.shape}")

        # 2. Load & merge time dimension
        df_temps = pd.read_csv(time_dim_path)
        logging.info(f"Loaded dim_temps → shape: {df_temps.shape}")

        if 'time_sk' not in df_facts.columns or 'time_sk' not in df_temps.columns:
            raise KeyError("'time_sk' missing from facts or dim_temps – cannot merge.")

        df = pd.merge(df_facts, df_temps, on='time_sk', how='left')
        logging.info(f"After facts ⊕ dim_temps merge → shape: {df.shape}")

        # 3. Drop columns we don't need
        cols_to_drop = ['weather_sk', 'event_sk', 'temperature', 'condition_text']
        df.drop(columns=cols_to_drop, errors='ignore', inplace=True)

        # 4. Dedup on natural business keys
        dedup_keys = ['time_sk', 'zone_sk', 'mode_sk']
        before = len(df)
        df.drop_duplicates(subset=dedup_

```

ML/actor1_ecologique/evaluate.py

```

import os
import json
import numpy as np
import pandas as pd
import joblib
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split
from datetime import datetime
import logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

def run_evaluation():
    """
    Central evaluation script for Actor 1.
    - Rebuilds the same temporal split used by model_regression.py (no data leakage).
    - Loads clustering silhouette score dynamically from the saved model.
    - Loads Prophet holdout MAE from the JSON written by model_timeseries.py.
    - Writes a single clean metrics_report.json for the professor.
    """
    logging.info("Starting central evaluation for Actor 1...")

    metrics = {}
    report_ok = True

    # -----
    # 1. Regression Evaluation
    # -----
    try:
        from data_loader import load_and_clean_data
        from feature_engineering import engineer_features

        df_clean = load_and_clean_data()
        df_reg, _, _ = engineer_features(df_clean)

        # Reload the exact feature list that was used during training
        feature_path = 'outputs/xgboost_features.pkl'
        if not os.path.exists(feature_path):
            raise FileNotFoundError(f"{feature_path} missing – run model_regression.py first.")
        features = joblib.load(feature_path)

        xgb_co2 = joblib.load('outputs/xgboost_co2.pkl')
        xgb_nrj = joblib.load('outputs/xgboost_nrj.pkl')

        # Reconstruct the temporal test set (last 8 months – matches model_regression v4)
        cutoff = df_reg['annee_mois_dt'].max() - pd.DateOffset(months=8)
        train_df = df_reg[df_reg['annee_mois_dt'] <= cutoff].copy()
        test_df = df_reg[df_reg['annee_mois_dt'] > cutoff].copy()

        if len(test_df) == 0

```

[ML/actor1_ecologique/export_predictions.py](#)

```

import os
import pandas as pd
import numpy as np
import joblib
from datetime import datetime
import logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

# -----
# Database config – set to None to skip the DB push entirely
# -----
DB_CONFIG = {
    'user': 'root',
    'password': '',
    'host': '127.0.0.1',
    'port': '3306',
    'database': 'outt',
}
ENABLE_DB_PUSH = False # Set True when DB is ready

def _push_to_mysql(df, table_name, if_exists='replace'):
    try:
        from sqlalchemy import create_engine
        u, pw, h, p, db = (
            DB_CONFIG['user'], DB_CONFIG['password'],
            DB_CONFIG['host'], DB_CONFIG['port'],
            DB_CONFIG['database'],
        )
        engine = create_engine(f"mysql+mysqlconnector://{u}:{pw}@{h}:{p}/{db}")
        df.to_sql(table_name, con=engine, if_exists=if_exists, index=False)
        logging.info(f" DB push OK → table `{table_name}`")
    except Exception as e:
        logging.warning(f" DB push skipped: {e}")

def generate_power_bi_export():
    """
    Produces all CSVs for Power BI from the trained models.

    Outputs
    -----
    outputs/regression_predictions.csv – full dataset predictions (historical)
    outputs/future_predictions.csv – synthetic future (2-year) scenarios
    outputs/forecast_aqi.csv – Prophet AQI/PM2.5/CO2/Energy (copy)
    outputs/cluster_labels.csv – zone cluster profile
    """
    logging.info("Initiating Power BI export v2...")
    os.makedirs('outputs', exist_ok=True)
    generated_at = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    # — Load models —
    feature_path = 'outputs/xgboost_features.pkl'
    if not os.path.exists(feature_path):
        raise FileNotFoundError("

```

ML/actor1_ecologique/feature_engineering.py

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
import joblib
import os
import logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

def engineer_features(df_clean):
    """
    Transforms clean data into ML-ready features.

    Strategy
    -----
    - Cyclical (sin/cos) month encoding → removes the 1<12 ordinal bias.
    - Lag features (1-month and 3-month) per (zone_sk, mode_sk).
    - Rolling 3-month mean of past values (shifted by 1 to avoid leakage).
    - carbon_intensity is a KPI OUTPUT column only – never used as a predictor.
    - Log1p transform on both regression targets to reduce right skew.
    - Returns df_reg (for XGBoost), df_cluster (for K-Means), df_prophet (raw aggs for Prophet).
    """
    logging.info("=== FEATURE ENGINEERING v3 ===")

    df = df_clean.copy()

    # — 0. Type safety —————
    df['annee'] = pd.to_numeric(df['annee'], errors='coerce').astype('Int64')
    df['mois'] = pd.to_numeric(df['mois'], errors='coerce').astype('Int64')
    df.dropna(subset=['annee', 'mois'], inplace=True)
    df['annee'] = df['annee'].astype(int)
    df['mois'] = df['mois'].astype(int)

    # — 1. Build datetime column —————
    df['annee_mois_dt'] = pd.to_datetime(
        df['annee'].astype(str) + '-' + df['mois'].astype(str).str.zfill(2) + '-01'
    )

    # — 2. Flag and drop zero targets —————
    initial_len = len(df)
    df['co2_kg'] = df['co2_kg'].replace(0, np.nan)
    df['aqi_index'] = df['aqi_index'].replace(0, np.nan)
    df['energie_kwh'] = df['energie_kwh'].replace(0, np.nan)
    df.dropna(subset=['co2_kg', 'aqi_index', 'energie_kwh'], inplace=True)
    logging.info(f"Dropped {initial_len - len(df)} rows with 0/NaN in targets → {len(df)} rows")

    # — 3. Categorical encoding —————

```

ML/actor1_ecologique/main.py

```

import logging
import time
import os
import sys

# -----
# Make sure the project root is on the path regardless of where you call this
# -----
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
os.chdir(os.path.dirname(os.path.abspath(__file__)))

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    handlers=[
        logging.StreamHandler(),
        logging.FileHandler('outputs/pipeline.log', mode='w', encoding='utf-8'),
    ]
)

def run_pipeline():
    """
    Master orchestrator for the Actor 1: Transition Écologique pipeline.

    Steps
    -----
    1. Data Loader          - load + dedup fact_impact_territorial.csv
    2. Feature Engineering - lags, cyclical encoding, log transforms
    3. Regression Models    - XGBoost (CO2 + Energy), temporal split, 5-fold CV
    4. Time Series          - Prophet per zone, 2-year horizon
    5. Clustering           - K-Means on zone profiles
    6. Evaluation           - unified metrics → metrics_report.json
    7. Export               - CSVs for Power BI + optional DB push
    """
    os.makedirs('outputs', exist_ok=True)

    print("\n" + "="*60)
    print("  ACTOR 1 – TRANSITION ÉCOLOGIQUE ML PIPELINE  v2")
    print("="*60 + "\n")

    start_time = time.time()
    pipeline_status = {}

    df_clean = None
    df_reg = None
    df_cluster = None
    df_prophet = None

    # — STEP 1: Data Loader —————
    try:
        logging.info(">>> STEP 1: Loading and Cleaning Data")
        from data_loader import load_and_clean_data
        df_clean = load_and_clean_data()
        logging.info(f"  Clean data shape: {df_clean.shape}")
        pipeline_status['1. Data Loader'] = f"✅ SUCCESS – {df_clean.shape[0]} rows"
    except Exception as e:
        logging.error(f"Data Lo

```

ML/actor1_ecologique/model_clustering.py

```

import os
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
import joblib
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

def train_pollution_clustering(df_cluster):
    """
    Groups the 10 zones into pollution+energy profiles using K-Means.

    Improvements over v1:
    - Aggregates to zone-level means BEFORE scaling (correct order).
    - Tries K=2,3,4; picks by silhouette score.
    - Saves cluster_labels.csv (expected by export_predictions.py).
    - Saves scaler + kmeans model to outputs/.
    - Dynamic cluster naming from centroid values.
    """
    logging.info("Initializing K-Means Clustering Pipeline v2...")
    os.makedirs('outputs', exist_ok=True)

    # — 1. Aggregate raw values to zone-level means —————
    raw_features = ['pm25', 'no2', 'co2_kg', 'energie_kwh']
    available = [f for f in raw_features if f in df_cluster.columns]

    if len(available) < 2:
        logging.error("Not enough clustering features found: {available}")
        return None

    zone_profiles = (
        df_cluster
        .replace({'co2_kg': {0: np.nan}, 'energie_kwh': {0: np.nan}})
        .dropna(subset=available)
        .groupby('zone_sk')[available]
        .mean()
        .reset_index()
    )

    # Attach zone names if available
    if 'zone_nom' in df_cluster.columns:
        zone_names = df_cluster[['zone_sk', 'zone_nom']].drop_duplicates()
        zone_profiles = pd.merge(zone_profiles, zone_names, on='zone_sk', how='left')
    else:
        zone_profiles['zone_nom'] = 'Zone_' + zone_profiles['zone_sk'].astype(str)

    n_zones = len(zone_profiles)
    logging.info(f"Clustering on {n_zones} unique zones.")

    if n_zones < 3:
        logging.error(f"Only {n_zones} zones – cannot cluster m

```

ML/actor1_ecologique/model_regression.py


```

import os
import pandas as pd
import numpy as np
import xgboost as xgb
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.model_selection import cross_val_score
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import joblib
import logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

# -----
# Features used within each per-mode model.
# mode_encoded is removed (constant within a mode group),
# mode_co2_mean is also removed (constant within a mode).
# -----
FEATURES_PER_MODE = [
    'zone_encoded',
    'annee', 'mois_sin', 'mois_cos',
    'pm25', 'no2', 'aqi_index',
    'co2_lag1', 'co2_lag3',
    'energie_lag1', 'energie_lag3',
    'aqi_lag1', 'pm25_lag1',
    'co2_roll3', 'energie_roll3', 'pm25_roll3',
]

# Features for the combined model (mode_encoded is the key discriminator)
FEATURES_COMBINED = [
    'zone_encoded', 'mode_encoded', 'mode_co2_mean',
    'annee', 'mois_sin', 'mois_cos',
    'pm25', 'no2', 'aqi_index',
    'co2_lag1', 'co2_lag3',
    'energie_lag1', 'energie_lag3',
    'aqi_lag1', 'pm25_lag1',
    'co2_roll3', 'energie_roll3', 'pm25_roll3',
]

def _temporal_split(df, test_months=8):
    cutoff = df['annee_mois_dt'].max() - pd.DateOffset(months=test_months)
    train_mask = df['annee_mois_dt'] <= cutoff
    test_mask = df['annee_mois_dt'] > cutoff
    return df[train_mask].copy(), df[test_mask].copy()

def _get_base_params():
    return {
        'n_estimators': 300,
        'max_depth': 4,
        'learning_rate': 0.03,
        'subsample': 0.8,
        'colsample_bytree': 0.8,
        'min_child_weight': 5,
        'gamma': 0.05,
        'reg_alpha': 0.3,
        'reg_lambda': 1.5,
        'objective': 'reg:squarederror',
        'rand':

```

ML/actor1_ecologique/model_timeseries.py

```

import os
import pandas as pd
import numpy as np
from prophet import Prophet
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import logging
import warnings
warnings.filterwarnings('ignore') # suppress Prophet's verbose Stan output

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

# How many future months to forecast beyond the training data
FORECAST_HORIZON = 24 # 2 years – gives future-year predictions for Power BI

def _tune_prophet(df_zone_monthly, target_col, n_holdout=3):
    """
    Tries two changepoint_prior_scale values and picks the one with lower
    MAE on the last n_holdout months. Returns the best fitted model.
    """
    if len(df_zone_monthly) <= n_holdout + 6:
        # Not enough data to do a meaningful holdout – just use default
        m = Prophet(
            yearly_seasonality=True,
            weekly_seasonality=False,
            daily_seasonality=False,
            changepoint_prior_scale=0.05,
        )
        m.fit(df_zone_monthly.rename(columns={'annee_mois_dt': 'ds', target_col: 'y'}))
        return m

    train_df = df_zone_monthly.iloc[:-n_holdout]
    val_df = df_zone_monthly.iloc[-n_holdout:]

    best_mae = float('inf')
    best_scale = 0.05

    for scale in [0.05, 0.3]:
        m = Prophet(
            yearly_seasonality=True,
            weekly_seasonality=False,
            daily_seasonality=False,
            changepoint_prior_scale=scale,
        )
        m.fit(train_df.rename(columns={'annee_mois_dt': 'ds', target_col: 'y'}))
        fut = m.make_future_dataframe(periods=n_holdout, freq='MS')
        fc = m.predict(fut).tail(n_holdout)
        mae = np.mean(np.abs(fc['yhat'].values - val_df[target_col].values))

        if mae < best_mae:
            best_mae = mae
            best_scale = scale

    # Refit on the full series with the best scale
    m_final = Prophet(
        yearly_seasonality=True,
        weekly_seasonality=False,

```

[ML/actor2_mobilites/data_loader.py](#)

```

import pandas as pd
import numpy as np
import logging
import os

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def load_mobility_data(
    fact_path=None,
    time_dim_path=None,
):
    """
    Loads fact_service_mobilite.csv, joins dim_temps to recover temporal columns
    (annee, mois, heure, jour_semaine, weekend, periode), drops irrelevant columns,
    and imputes ETL artefact values.

    Key decisions
    -----
    - NO dedup needed (fan-out issue was specific to fact_impact_territorial).
    - Drops weather_sk (95% NULL), event_sk (99% NULL), bak_* columns.
    - vitesse_kmh == 0 treated as ETL artefact → replaced with NaN for later imputation.
      (bak_vitesse_kmh confirms original values were 0 → NULL in trafic join)
    - stress_1_5 nulls imputed with zone median in feature_engineering.
    - Prints full null / zero report so downstream scripts start from a verified state.
    """
    logging.info("=== DATA LOADER v1 – Actor 2 Mobilités ===")

    if fact_path is None:
        fact_path = os.path.join(BASE_DIR, 'fact_service_mobilite.csv')
    if time_dim_path is None:
        time_dim_path = os.path.join(BASE_DIR, 'dim_temps.csv')

    try:
        # 1. Load facts
        df_facts = pd.read_csv(fact_path)
        logging.info(f"Loaded facts → shape: {df_facts.shape}")

        # 2. Load & merge time dimension (restores annee, mois, heure, etc.)
        df_temps = pd.read_csv(time_dim_path)
        logging.info(f"Loaded dim_temps → shape: {df_temps.shape}")

        if 'time_sk' not in df_facts.columns or 'time_sk' not in df_temps.columns:
            raise KeyError("'time_sk' missing from facts or dim_temps.")

        df = pd.merge(df_facts, df_temps, on='time_sk', how='left')
        logging.info(f"After fact ⊕ dim_temps merge → shape: {df.shape}")

        # 3. Drop columns we don't need
        cols_to_drop = [
            'weather_sk', 'ev

```

ML/actor2_mobilites/evaluate.py

```

import os
import json
import numpy as np
import pandas as pd
import joblib
import logging
from datetime import datetime
from sklearn.metrics import (
    mean_squared_error, mean_absolute_error, r2_score,
    f1_score, roc_auc_score, precision_score, recall_score, confusion_matrix
)

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

GENERATED_AT = datetime.utcnow().strftime('%Y-%m-%d %H:%M:%S UTC')

def evaluate_actor2(df_ml=None):
    """
    Central evaluation hub for Actor 2.

    - Reloads saved models and reproduces the exact same test split.
    - Reports Regression (RMSE, MAE, R²), Classification (F1, AUC, Precision, Recall,
      confusion matrix at two thresholds), and KPI (Taux de Ponctualité).
    - Reads Prophet holdout metrics from the cached classification_report.json if available.
    - Writes outputs/metrics_report.json.
    """
    logging.info("Starting central evaluation for Actor 2...")
    metrics = {
        'metadata': {
            'actor': 'Actor 2 - Directeur Mobilités',
            'execution_date': GENERATED_AT,
        }
    }

    # — 1. Load data if not provided —————
    if df_ml is None:
        from data_loader import load_mobility_data
        from feature_engineering import engineer_features
        df_raw = load_mobility_data()
        df_ml, _ = engineer_features(df_raw)

    # — 2. Regression - Passenger Load —————
    try:
        from sklearn.model_selection import train_test_split
        reg_model = joblib.load('outputs/xgboost_charge.pkl')
        reg_features = joblib.load('outputs/xgboost_charge_features.pkl')

        # Load target encoding maps (needed for line_charge_mean, zone_hour_charge)
        charge_enc = joblib.load('outputs/charge_encoding.pkl') if
os.path.exists('outputs/charge_encoding.pkl') else None

        # Reproduce the exact base columns and split
        base_cols = ['zone_encoded',

```

[ML/actor2_mobilites/export_predictions.py](#)

```

import os
import logging
import numpy as np
import pandas as pd
import joblib
from datetime import datetime

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

GENERATED_AT = datetime.utcnow().strftime('%Y-%m-%d %H:%M:%S UTC')

LOW_THRESHOLD = 0.20
MEDIUM_THRESHOLD = 0.50

def _risk_label(p):
    if p < LOW_THRESHOLD:
        return 'Low'
    elif p < MEDIUM_THRESHOLD:
        return 'Medium'
    return 'High'

def export_predictions(df_ml, df_ts=None):
    """
    Generates three Power BI-ready CSVs:
    1. load_predictions.csv - predicted charge_estimee per trip row
    2. cancellation_risk.csv - annule_pred + annule_proba + risk_label per trip row
    3. congestion_forecast.csv - already written by model_timeseries; just verified here

    All files include zone_sk + line_sk as join keys for Power BI.
    """
    logging.info("=== EXPORT PREDICTIONS - Actor 2 ===")
    os.makedirs('outputs', exist_ok=True)

    # --- Load models ---
    reg_model = joblib.load('outputs/xgboost_charge.pkl')
    reg_features = joblib.load('outputs/xgboost_charge_features.pkl')
    clf_model = joblib.load('outputs/xgboost_cancellation.pkl')
    clf_features = joblib.load('outputs/xgboost_cancellation_features.pkl')

    # Load charge target encoding maps
    charge_enc = joblib.load('outputs/charge_encoding.pkl') if
os.path.exists('outputs/charge_encoding.pkl') else None

    # --- 1. load_predictions.csv ---
    # Apply target encoding maps if available
    if charge_enc is not None:
        gm = charge_enc['global_mean']
        df_reg_in = df_ml[[c for c in ['zone_encoded', 'line_encoded', 'mode_encoded', 'hour',
                                     'rush_hour', 'is_weekend', 'congestion_index',
                                     'vitesse_kmh', 'temps_trajet_min'] if c in
df_ml.columns]].copy()
        for

```

[ML/actor2_mobilites/feature_engineering.py](#)

```

import pandas as pd
import numpy as np
import logging
import os

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

def engineer_features(df_raw):
    """
    Transforms clean mobility data into ML-ready feature sets.

    Transformations
    -----
    1. Imputation - vitesse_kmh + temps_trajet_min + stress_1_5 with zone median;
                   global median fallback for zones that are entirely null.
    2. Time feats - hour (int 0-23), rush_hour (binary), is_weekend (binary).
    3. Encoding   - zone_sk / line_sk / mode_sk as category codes.
    4. retard_flag - alias for annule (only cancellation signal available).
    5. Imbalance  - logs the dynamic scale_pos_weight for the classifier.

    Returns
    -----
    df_ml      : full feature-engineered DataFrame (used by regression + classification)
    df_ts      : minimal DataFrame for Prophet (zone_sk, date, congestion_index)
    """
    logging.info("=== FEATURE ENGINEERING v1 - Actor 2 Mobilités ===")

    df = df_raw.copy()

    # — 1. Zone-median imputation —————
    cols_to_impute = ['vitesse_kmh', 'temps_trajet_min', 'stress_1_5']
    for col in cols_to_impute:
        if col not in df.columns:
            continue
        n_null_before = df[col].isnull().sum()
        if n_null_before == 0:
            continue
        # Per-zone median, then global fallback
        df[col] = df.groupby('zone_sk')[col].transform(
            lambda x: x.fillna(x.median())
        )
        df[col] = df[col].fillna(df[col].median())
        n_null_after = df[col].isnull().sum()
        logging.info(f"Imputed '{col}': {n_null_before} → {n_null_after} nulls")

    # — 2. Time features —————
    if 'heure' in df.columns:
        heure_s = df['heure'].astype(str).str.strip()
        if heure_s.str.contains(':').any():
            # "HH:MM:SS" - extract hour directly from first se

```

[ML/actor2_mobilites/main.py](#)

```

"""
Actor 2 – Directeur Mobilités (RATP / Île-de-France Mobilités)
=====
ML Pipeline Orchestrator

Steps
-----
1. Data Loader          – fact_service_mobilite.csv + dim_temps merge
2. Feature Engineering – imputation, encoding, time features, Prophet aggregation
3. Regression           – XGBoost → predict charge_estimee (passenger load)
4. Classification       – XGBoost → predict cancellation risk (annule)
5. Time Series          – Prophet → 30-day daily congestion forecast per zone
6. Evaluation           – Central metrics report (JSON)
7. Export               – load_predictions.csv, cancellation_risk.csv, congestion_forecast.csv
"""

import os
import sys
import time
import logging
from datetime import datetime

# Always run relative to this file's directory (important when invoked from actor1 venv)
os.chdir(os.path.dirname(os.path.abspath(__file__)))

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    handlers=[
        logging.StreamHandler(sys.stdout),
        logging.FileHandler('outputs/pipeline.log', mode='w', encoding='utf-8'),
    ],
)
os.makedirs('outputs', exist_ok=True)

BANNER = """
=====
ACTOR 2 – DIRECTEUR MOBILITÉS ML PIPELINE
=====
"""

def _step(name, fn, *args, **kwargs):
    """Runs a pipeline step with timing and error isolation."""
    start = time.time()
    logging.info(f">>> STEP: {name}")
    try:
        result = fn(*args, **kwargs)
        elapsed = time.time() - start
        logging.info(f"    ✓ {name} – {elapsed:.1f}s")
        return result, True
    except Exception as e:
        elapsed = time.time() - start
        logging.error(f"    ✗ {name} FAILED ({elapsed:.1f}s): {e}")
        import traceback
        logging.debug(traceback.format_exc())
        return None, False

def run_pipeline()

```

[ML/actor2_mobilites/model_classification.py](#)

```

import os
import json
import numpy as np
import pandas as pd
import xgboost as xgb
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    f1_score, roc_auc_score, precision_score, recall_score,
    confusion_matrix, precision_recall_curve
)
import joblib
import logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

FEATURES = [
    'zone_encoded', 'line_encoded', 'mode_encoded',
    'hour', 'rush_hour', 'is_weekend',
    'congestion_index', 'charge_estimee',
]
TARGET = 'retard_flag' # alias for annule in feature_engineering

# Risk label thresholds
LOW_THRESHOLD = 0.20
MEDIUM_THRESHOLD = 0.50

def _risk_label(proba: float) -> str:
    if proba < LOW_THRESHOLD:
        return 'Low'
    elif proba < MEDIUM_THRESHOLD:
        return 'Medium'
    else:
        return 'High'

def train_cancellation_model(df_ml):
    """
    Trains XGBoost classifier to predict trip cancellation probability (annule=1).

    Key design decisions
    -----
    - scale_pos_weight = neg/pos (dynamic) → handles the 2% positive class imbalance
      without SMOTE. Equally effective for tree models and simpler.
    - Outputs both binary (0/1) and probability score per trip.
    - Evaluates at default threshold (0.5) AND finds the F1-maximising threshold.
    - Saves confusion matrix, ROC curve, and precision-recall curve for the professor.
    - risk_label column (Low/Medium/High) ready for Power BI conditional formatting.
    """
    logging.info("=== CLASSIFICATION – Trip Cancellation Risk ===")
    os.makedirs('outputs', exist_ok=True)

    # — 1. Feature selection —————
    available = [f for f in FEATURES if f in df_ml.columns]
    missing = set(FEATURES) - set(available)
    if missing:
        logging.warning(f"Missing features (skipped): {missing}")

    if TARGET not in

```


Annexe B — Guide d'Installation et de Déploiement

B.1 Prérequis

- Python 3.11+
- Node.js 18+
- Docker 24+
- Prometheus binary
- Grafana service
- n8n (npm i -g n8n)

B.2 Installation

```
git clone https://github.com/HeniRidene/Esprit-PABI-4ERPBI5-2526-BITribe.git
cd Esprit-PABI-4ERPBI5-2526-BITribe
python -m venv .venv && .venv\Scripts\activate
pip install -r ml_api_2/requirements.txt
cd urban-mobility-website && npm install && cd ..
```

B.3 Lancement — STARTUP.ps1

```
# Le script STARTUP.ps1 lance 8 services :
# 1. MLflow server (port 5000)
# 2. FastAPI (port 8000)
# 3. n8n (port 5678)
# 4. Prometheus (port 9090)
# 5. Grafana (port 3001)
# 6. Streamlit (port 8501)
# 7. Next.js dev (port 3000)
# 8. Alerting daemon (alerting.py)
.\STARTUP.ps1
```

B.4 Training — TRAIN_MODELS.ps1

```
.\TRAIN_MODELS.ps1
# Execute sequentially:
# 1. ML/actor1_ecologique/main.py (7 steps)
# 2. ML/actor2_mobilites/main.py (7 steps)
# 3. ML/actor3_securite/main.py (7 steps)
# Validates: 8 PKL files generated
```

B.5 Déploiement Docker

```
cd ml_api_2
docker-compose up --build
# Services: api (8000) + mlflow (5000)
# Volumes: actor outputs mounted
```

Annexe C — Documentation API

Méthode	Endpoint	Description	Corps
GET	/health	Liveness probe	—
GET	/models	Liste acteurs/tâches	—
GET	/metrics	Métriques Prometheus	—
GET	/drift	Détection drift 3 acteurs	—
POST	/predict	{actor, task, features}	PredictRequest
POST	/retrain	Retraining 3 acteurs	—
POST	/trigger-n8n-predict	Trigger webhook n8n	PredictRequest
POST	/trigger-n8n-retrain	Trigger workflow n8n	—
POST	/save-n8n-result	Sauvegarde résultat n8n	dict

C.1 Exemple de Requête

```
POST /predict
Content-Type: application/json

{
  "actor": "actor1",
  "task": "co2",
  "features": {
    "zone_sk": 1,
    "mode_sk": 2,
    "annee": 2024,
    "mois": 6,
    "aqi_index": 80,
    "pm25": 35
  }
}

Response 200:
{
  "status": "success",
  "actor": "actor1",
  "task": "co2",
  "result": 245.67,
  "latency_ms": 14.3
}
```

Annexe D — Glossaire

Terme	Définition
Actor	Module décisionnel associé à un rôle métier spécifique
Drift	Changement dans la distribution des données ou des prédictions
Feature Engineering	Transformation de données brutes en features exploitables par un modèle ML
Inférence	Processus de génération de prédictions par un modèle entraîné
Lazy Loading	Chargement d'un modèle uniquement lors de la première utilisation
Pipeline	Séquence automatisée d'étapes de traitement de données
PKL	Format de sérialisation Python (pickle/joblib)
Scrape	Collecte automatique de métriques par Prometheus
Webhook	Endpoint HTTP appelé automatiquement lors d'un événement

Plateforme Intelligente de Mobilité Urbaine

ESPRIT — PABI 4ERPBI5 — 2025/2026

Ce rapport a été généré automatiquement à partir de l'analyse complète du code source du projet.